BITS Pilani
DIGITAL
Excellence made yours

Data Engineering for AI Systems
Module 2: Week 4

Balu Gopalakrishna Pillai

Recap of Week 3

Sections

1. Introduction: Theme for the Week
2. Data Collection
3. Data Collection as an Engineering Problem
4. Data Preprocessing
5. Data Leakage and Leakage Patterns
6. Data Validation and Categories of Data Validation
7. Activity: Building A Data Pipelines

Section 1 : Theme of the Week

Data Pipelines

Make data trustworthy before it becomes dangerous.

Data pipelines are production systems:

- They fail
- They drift
- They introduce risk
- When they break, they rarely announce

Goal for the Week

- Teach you how to make data trustworthy before it becomes dangerous
- How we do it: Data collection, preprocessing and validation

Section 2: Data Collection

What is Data Collection?

- Systemic processing of gathering data from various sources
- Accurate, reliable and ethical
- Data quality directly impacts model behavior, fairness, reliability and long term stability of the system
- Poor data — drift, bias, cascading failure, legal risk, loss of trust

Section 3: Data Collection as an Engineering Problem

Data Sources, Formats and Failure Modes

Sources:

- APIs
- Sensors and devices
- Application logs
- Third party providers
- Human input
- Legacy databases

Data Sources, Formats and Failure Modes

Data Source	Typical Latency	Reliability Characterstics	Common Failure Modes
APIs (Internal/External)	Low => Medium. (Ms to seconds)	Depends on provider SLA, network stability, auth cycle etc	Rate limiting, schema changes, silent field removal, partial responses, auth expiry etc
Application Logs	Near real time =>Batch delay (seconds to minutes)	High when pipeline is well engineered	Log loss under high load, format drift, missing fields, duplicate ingestion, out-of-order arrival etc
Human Inputs (Forms, Annotation, Manual Entry)	Very high and variable (minutes => days =>weeks)	Low to medium, Affected by training, fatigue, interpretation and workflow design.	Inconsistent labels, missing values, delayed entry, subjective bias etc
Sensors/IoT/Devices	Ultra-low for streaming (ms)	Medium in controlled environments, low in field deployments	Data gaps due to network loss, calibration drift, stuck values, duplicate burst after reconnect
Third Party Data	Medium => High	Medium. Contract-bound	Delivery delays, schema changes without notice, licensing restrictions etc
Legacy Database	High for build extraction	Medium, stable structure but fragile integration	Incomplete extract, undocumented business logic, encoding issues

Sampling Bias at Collection Time

What you decide to collect - or fail to collect

For example:

1. Only collecting tickets from premium users
2. Only logging successful transactions
3. Only labelling cases that were escalated
4. Only storing data that passed earlier filters

The dataset looks complete. The schema validates. Nothing crashes.
But really captured is only the slice of the world.
And models are extremely confident when trained on incomplete truth.

Distribution Mismatch: Training vs Production Reality

Training distribution does not match production distribution

Your model trains on:

- Clean
- Historical
- Backfilled
- Carefully

Production sends:

- Missing values
- Edge cases
- Behavior changes
- New categories
- Human Inconsistency

The model did not degrade, it is seeing the world you never showed it.

Hidden Upstream Changes

The hardest problem to detect!

Hidden filters upstream.

- Someone added a WHERE clause six months ago.
- Someone changed an API default.
- Someone dropped nulls “to fix a dashboard”.
- Someone deduplicated records incorrectly.

No one told the ML team

- Your pipeline still runs
- Your metrics will still compute
- But the meaning of the data has changed
- You inherit the consequence - without context

Data Collection - Tools and Best Practices

Tools:

- Ingestion and Integration - Airbyte, FiveTran
- Streaming - Kafka
- Web and Unstructured data - Scrapy, Playwright, Beautifulsoup

Best Practices:

- Define the Data Contract First - before collecting: Schema, ownership, etc
- Collect with the ML Use Case in Mind: Instead of ‘store everything’, design for training, validation, monitoring, etc
- Capture metadata and Lineage: Store source, timestamp, version, etc. for reproducibility, audit, etc
- Incremental Collection: Avoid full reloads, use watermarks for faster pipelines
- Data quality at entry point: validate at ingestion - null checks, range checks, etc
- Handle PII and Compliance: Mask sensitive fields - GDPR, HIPPA

Key Takeaways

- If you don’t control collection, you don’t control outcomes.
- If you don’t validate inputs, you can’t trust outputs.
- And if your data pipeline isn’t engineered, your AI system isn’t either.

Break

Section 4: Data Preprocessing

What is Data Preprocessing?

Preprocessing is not cleaning.

Cleaning:

- Asks: What is wrong with the data?
- Removes errors
- Reactive
- Can outsource

Preprocessing:

- Asks: How must this data be shaped so the system can reason about it consistently?
- Imposes structure
- Architectural
- Cannot outsource, why? Preprocessing defines how your model interprets reality

Preprocessing Answers

Questions like:

What counts as “recent”?

What counts as “similar”?

What counts as “absence”?

What counts as “zero”?

What counts as a single event versus a pattern?

Preprocessing Methods and Tools

Where semantics enter the AI system

Common preprocessing methods:

- Normalisation and scaling
- Tokenization of text
- Handling missing values
- Encoding categorical variables
- Feature construction and aggregation
- Time-windowing and alignment
- Deduplication logic
- Label construction

These do not just prepare the data; they also define the model's vocabulary.

The model never sees the raw data; only the world you constructed by preprocessing

Preprocessing: Normalisation and Scaling

Transforming into a common value range

Why it matters
- Many LLM models are distance-based or gradient-based
- Large-value features bias the model
- Improves convergence speed in deep learning

Example
Income: 10,000 – 1,000,000
Age: 18 – 60
Without scaling → model thinks income is more important.

Common techniques
- Min–Max scaling → range [0,1]
- Standardization → mean = 0, std = 1
- Robust scaling → resistant to outliers
- Log scaling → for skewed data

Min-Mac Scaling (0->1 range)

Formula X-scaled = (X - Xmin) / (X max - X min)

For 10,000, X scaled = (10000 - 10000) / (1000000 - 10000) = 0.0
50,000, X scaled = 0.495
1000000, X scaled = 1.0

For 18, X scaled = (18 - 18) / (60 - 18) = 0.0
25, X scaled = (25 - 18) / (60 - 18) = 0.167
40, X scaled = (40 - 18) / (60 - 18) = 0.524
60, X scaled = (60 - 18) / (60 - 18) = 1

Both results are in 0 -> 1 range, income no longer dominates.
Age & income contribute proportionately

Preprocessing: Tokenization of Text

BITS Pilani DIGITAL
Excellence made yours

Breaking texts into units (tokens) for model consumption

Why it matters: Models don't understand raw text — they understand numbers.

Types

- Word tokenization
- Subword tokenization (BPE, WordPiece) → used in LLMs
- Character tokenization

Example

"Data pipelines are powerful"

Word tokens → 4

Subword tokens → may become 6–8

Defines: Cost, Context length, Semantic representation

Tools: Hugging Face tokenizers, spaCy, NLTK, OpenAI / sentencepiece tokenizers

Preprocessing: Handling Missing Values

Managing null or unavailable data

Why it matters: Most ML models cannot handle missing values directly.
Example
Loan application:
Missing income → not random → high risk.
Strategies

1. Drop rows (only if small %)
2. Mean/median / mode imputation
3. Forward fill / backwards fill (time series)
4. Model-based imputation
5. Special “unknown” category

AI system view: Missing data can be:

- A signal (e.g., user inactive)
- A data quality issue

Tools: Pandas / PySpark fillna, Scikit-learn imputers, Feature store transformations

Encoding Categorical Values

Converting categories into numerical forms

Why it matters: Models require numerical input.
Example
City:
One-hot → Bangalore, Mumbai, Delhi
But 10,000 cities? → use embeddings or hashing.

Techniques

- One-hot encoding → low cardinality
- Label encoding → ordered categories only
- Target encoding → high cardinality
- Embeddings → deep learning and recommender systems
- Hashing → large-scale streaming systems

Tools: Scikit-learn encoders, Spark ML, TensorFlow feature columns

Feature Construction and Aggregation

Creating new features from raw data

Why it matters: Models learn from representations, not raw data.
Example:
Raw → transaction logs
Feature → avg spend last 30 days
Types:

- Ratios (revenue per user)
- Counts (transactions per day)
- Statistical aggregates (avg, max, std)
- Domain features

Tools: SQL, Spark, Feature stores (Feast, Tecton)

Time Windowing and Alignment

Create feature over a specific historical time window

Why it matters: AI systems operate in time — leakage risk is high.
Examples:

- Last 7 days activity
- Rolling averages
- Session-based features

Critical for:

- Fraud detection
- Recommendations
- Forecasting

Tools: Spark window functions, Flink, Feature store time-travel

Duplication Logic

Removing repeated or duplicated records

Why it matters:
- Bias the model
- Inflate feature importance
- Distort evaluation metrics

Types
- Exact duplicates
- Entity duplicates (same user, multiple IDs)
- Event duplicates (retries in streaming)

Techniques
- Primary key checks
- Hashing
- Fuzzy matching
- Window + rank in SQL

AI impact
LLM fine-tuning on duplicated data → overfitting to repeated patterns.

Label Construction

Creating the target variable for supervised training

Why it matters: Bad labels → bad models, even with perfect features.

Examples

Fraud model: Label = transaction marked fraud within 30 days

Churn model: Label = no login for 60 days

Tools

- SQL + business rules
- Annotation platforms
- Weak supervision (Snorkel)

A Practical Mental Model

Preprocessing defines the contract between the data and the model.

It answers three questions:

1. **Representation — How is reality encoded?**
   Scaling, tokenization, embeddings, categories
2. **Validity —** What inputs are allowed to exist?
   Null handling, bounds, and schema enforcement
3. **Causality —** What information is allowed at prediction time?
   Temporal discipline, leakage prevention

If we get this right, modeling becomes easier, else no model can save us.

Section 5: Data Leakage and Leakage Patterns

Data Leakage

The most dangerous failure!

- Leakage occurs when information from the future — or from outside the prediction boundary.
- Sneaks into training
- And leakage is dangerous because it doesn’t degrade performance.
- It inflates it.
- The model appears brilliant until it meets reality.

Common Data Leakage Patterns

BITS Pilani DIGITAL
Excellence made yours

That look innocent

- Global Normalisation
- Outcome-aware feature engineering
- Temporal aggregation errors
- Data joins without time boundaries

Global Normalisation

- You normalize using statistics computed on the entire dataset.
- That means your training data “knows” about distributions that only exist in the future.
- Correct approach: Compute normalization only on the training window, then apply forward.

Outcome-Aware Feature Engineering

- You accidentally include signals derived from the label.
- Example:
  - Predicting churn using “account closed date”
  - Predicting fraud using features created during the investigation
- The model is no longer predicting.
- It’s reverse-engineering.

Temporal Aggregation Errors

- You compute rolling averages that include events after the prediction timestamp.
- This violates causality.
- Production systems live forward in time.
- Your preprocessing must too.

Data Joins Without Time Boundaries

A classic real-world mistake:

- You join customer records with a table that was updated later — silently injecting future corrections into historical training rows.
- The join works technically.
- But logically, you just trained on information that didn’t exist yet.

Why Data Leaks are Hard to Detect

- Leakage doesn’t throw errors.
- It creates great validation metrics.
- That’s why it survives code reviews.
- The only reliable defense is this question: “Could this exact transformation have been executed at the moment of prediction?”
- If the answer is no, it doesn’t belong in preprocessing.

Missing Data is Not Empty Data

- Treating missing values as noise instead of signal.
- In real systems, missingness is often **behavioral**.
- A missing field can mean:
  - A user refused to answer
  - A sensor failed
  - A process was skipped
  - A transaction never happened
  - A high-risk case avoided automation
- When you blindly impute values, you erase that story.
- Instead, mature preprocessing often does two things:
  - Imputes a value
  - Adds a **missingness indicator feature**

Because "unknown" is sometimes the most predictive state.

Preprocessing: Key Insights

Preprocessing decisions encode assumptions.

Those assumptions become behavior.

Models don’t understand your intentions.

They operationalize your transformations.

So when we design preprocessing pipelines, we’re not cleaning data.

We’re defining the rules of perception for the system we’re building.

BITS Pilani
DIGITAL
Excellence made yours

Section 6: Data Validation and Categories of Data Validation

Data Validation

Is a contract, not a safety net!

A Contract Between Data and Model

Contracts

A contract says:

“If the data does not satisfy these conditions, it is not allowed to enter the system.”

In traditional software, we enforce contracts with types, interfaces and tests.

In ML systems, we must do the same — but for data.

Because, unlike code, data can change without a deployment.

And when data changes silently, models fail silently.

Why Validation Must be Continuous

Many teams validate data once, during dataset creation.
But production data is not static.

It evolves because:

- Users change behavior
- Upstream services change logic
- Sensors degrade
- Business rules get updated
- Logging pipelines get refactored
- New categories appear without warning

So validation is not a gate you pass once.
It's a monitoring discipline that runs every time data flows.

Four Categories of Data Validation

Schema checks - for **Structural Integrity**

Statistical checks - for **Behavioral Drift Detection**

Semantic check - for **Domain Reality Enforcement**

Temporal check - for **Causality Protection**

Data Validation: Schema checks

These are the most familiar checks.

- Do required fields exist?
- Are types correct?
- Are formats valid?
- Are values within allowed ranges?
- Did a column suddenly disappear or get renamed?

Schema validation prevents obvious breakage — the kind that would crash code.
But schema checks alone only tell you: “The data is shaped correctly.”
They do not tell you whether the data is meaningful.
That’s where the next layer begins.

Data Validation: Statistical Checks

Includes checks like:

- Has the distribution changed?
- Are means, variances or quantiles drifting?
- Are category frequencies shifting?
- Did a rare value suddenly become common?
- Did a dominant signal disappear?

Most so-called “model drift” is actually undetected data drift.
The model is behaving exactly as trained.
The world it’s seeing has changed.
Statistical validation lets you detect that change early — before performance collapses.

Data Validation: Semantic Check

A schema tells you that data is valid syntactically.
Semantics tell you whether it makes sense.

Examples:

- Is age negative?
- Is discharge time before admission?
- Is revenue recorded for a cancelled order?
- Does this label contradict known business rules?
- Is a device reporting values outside physical possibility?

These checks require domain knowledge.
They cannot be autogenerated.
They must be designed with the people who understand the system being modeled.
Now data validation becomes **collaboration between engineering & subject-matter experts.**

Data Validation: Temporal Checks

Temporal validation is the most overlooked — and the most dangerous.

You must verify:

- Are timestamps ordered correctly?
- Is data arriving late or out of order?
- Are backfills contaminating real-time pipelines?
- Are we accidentally mixing future and past?
- Is feature generation respecting prediction-time boundaries?

Temporal bugs are notoriously hard to debug because nothing crashes.
The system still runs.
It just becomes logically inconsistent.
And once causality is violated, evaluation metrics become meaningless.

Data Validation for Decisions

Should Trigger Decisions, Not Just Alerts

A mature system defines actions:

- Reject the batch
- Quarantine anomalous records
- Fallback to safe defaults
- Trigger retraining investigation
- Alert with context, not noise

Validation must influence system behavior — not just describe it.

Data Validation for Feedback loop

Creates trustable feedback loop

When validation is implemented correctly, it enables something critical: **safe iteration.**

You can:

- Add new data sources
- Update pipelines
- Retrain models
- Change feature logic

Because the contracts will tell you when you've broken something.
Without validation, every change becomes a risk.
With validation, change becomes measurable.

Data Validation: Key Insight

- If you don’t write validation rules, you are implicitly trusting everything upstream.
- And upstream always breaks.
- Reliable ML systems are not defined by better models.
- They’re defined by **explicitly enforced assumptions about data.**

BITS Pilani DIGITAL
Excellence made yours

Section 7: Data Pipelines & Three Pillars of Data Pipelines

Data Pipelines for Reproducibility

BITS Pilani DIGITAL
Excellence made yours

It is not a convenience; it is accountability!

Let's talk about reproducibility.

- If you can't reproduce your dataset, you can't debug your model.
- And if you can't debug your model, you don't have an engineering system.
- You have a collection of experiments that happened to work once.
- Reproducibility is what turns machine learning from exploration into **reliable production behavior.**

Why Reproducibility Fails in Real Systems

Discovery Vs Production

In early-stage projects, datasets are often assembled like this:

- A notebook pulls data
- Someone applies transformations interactively
- Intermediate files get saved locally
- A CSV is exported and shared
- The model is trained
- The notebook is never run the same way again

This works for discovery. It completely fails for production.
Because when performance drops three months later, no one can answer:
What exactly did we train on? Why does this happen?

Reproducibility Requires Engineering

It needs structural guarantees:

- The same inputs must produce the same output
- Every transformation must be encoded in code
- Every run must be traceable
- Every dataset must be reconstructable

That’s why data pipelines must be treated like software systems - not always like workflows

Three Pillars of Data Pipelines

Reproducibility requires three things to work together:

- Deterministic Preprocessing
- Parametrized Pipelines
- Explicit Data Lineage

Data Pipelines: Deterministic Preprocessing

The same raw data and configuration should produce the same dataset!

That means:

- No hidden randomness
- No manual edits
- No “run this cell if needed” logic
- No environment-dependent transformations

Determinism makes failures diagnosable.
Without it, every rerun becomes a different experiment.

Data Pipelines: Parametrized Pipelines

Should be rewritten for new experiments, would be parametrized

For example:

- Date ranges
- Feature windows
- Filtering rules
- Sampling strategies
- Model targets

This allows you to change behavior **without changing logic,** which is essential for controlled experimentation. When parameters change, you know what changed. When code changes, everything becomes suspect.

Data Pipelines: Explicit Data Lineage

Data Lineage - The foundation of auditability!

You should always be able to answer:

- Which raw data produced this dataset?
- Which code version generated it?
- Which parameters were used?
- Which schema version applied?
- At what timestamp was it built?
- Was it backfilled or real-time derived?

Reproducibility

Turns experiments to evidence!

Without reproducibility, experiments are stories.
With reproducibility, experiments become evidence.
You can compare runs.
You can isolate causes.
You can roll back safely.
You can validate claims.

This is what allows ML systems to operate in regulated, high-stakes or continuously evolving environments.

Operational Artifacts of Production Pipeline

Versioned — changes tracked like application code

Replayable — able to rebuild historical datasets exactly

Auditable — explainable to engineers, stakeholders and regulators

Observable — monitored for failures and drift

Automated — not dependent on manual execution

- Anything other than a reproducible pipeline is hope-driven engineering.
- And hope is not a strategy for a system that learn from data. Reliable ML does not start with better models. But with data pipelines you can trust, replay and explain

Section 7: Activity – Building a Data Pipeline

Instructions

Case Study: Mid-Tier Hospital AI System (shared in Week 3)

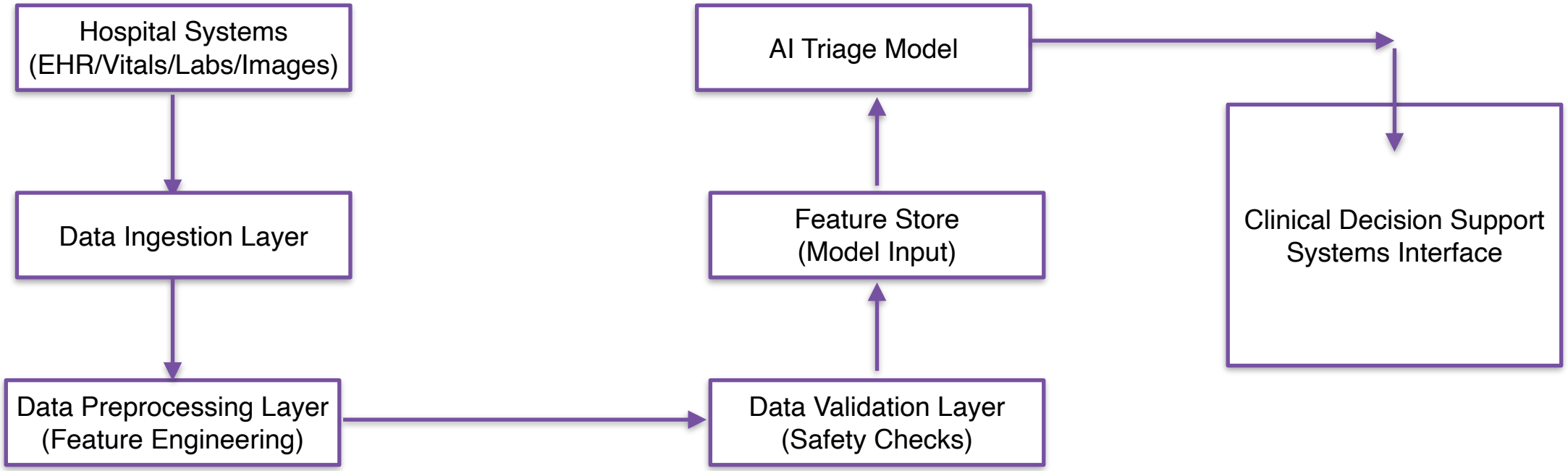
In today's lab, you will:

- Build a basic ingestion and preprocessing pipeline
- Add validation checks
- Deliberately introduce a failure
- And observe how it propagates downstream

Deliverable:

- A reproducible pipeline
- A valid report
- A known data risk section

High-Level Pipeline



Data Sources

Source	Data Type	Example
Electronic Health Record	Patient history	past diagnoses
Triage intake	Symptoms & nurse notes	chest pain
Vital monitoring	Physiological signals	HR, BP
Laboratory systems	Test results	blood counts
Radiology systems	Imaging reports	CT scan

Healthcare interoperability standard typically used:

- FHIR (Fast Healthcare Interoperability Resources)
- HL7 v2 messaging standard

These standards allow data exchange between hospital systems.

Data Ingestion Layer

Responsibilities

- Connect to hospital systems
- Normalize formats
- Timestamp all incoming events
- Route data to the pipeline

Typical ingestion architecture:

Hospital Systems (FHIR / HL7 APIs) =>
Streaming Queue =>
Raw Data Storage

Possible tools:

1. Streaming : Kafka
2. Integration : FHIR API gateway
3. RawStorage : Data Lake (S3 bucket/ HDFS)

Example incoming data event:

```
patient_data = {
    "patient_id": 98721,
    "arrival_time": "2026-03-05T18:22",
    "heart_rate": 112,
    "blood_pressure": "90/60",
    "symptom": "severe abdominal pain",
    "triage_notes": "vomiting"
}
```

Preprocessing Layer

Tasks

- Clean missing values
- Normalize units
- Encode symptoms
- Extract information from clinical notes
- Aggregate recent vitals

Example feature set:

- age
- heart_rate
- systolic_bp
- respiratory_rate
- temperature
- symptom_vector
- comorbidity_score

Raw Field	Process	Feature
Heart rate	normalize	HR_zscore
Blood pressure	split	systolic, diastolic
Symptoms	NLP extraction	symptom_vector
Notes	clinical NLP	condition indicators

Clinical text processing may use medical ontologies like:

- SNOMED CT
- ICD-10

Data Validation Layer

Critical for safety

Validation checks

Check	Purpose
Missing patient ID	prevent orphan records
Heart rate range	detect sensor errors
Blood pressure format	ensure valid readings
Timestamp order	detect time anomalies

Example rules:

- heart_rate must be between 30 and 220
- temperature must be between 30°C and 45°C
- age must be between 0 and 120

If validation fails:
Invalid_record => quarantine dataset => alert data team

Feature Store

for consistent model usage.

Purpose:

- reusable ML features
- consistent training and inference
- traceability

Feature store example structure:

patient_features

- vitals
- demographics
- symptom_features
- comorbidity_scores

Each record contains:

- patient_id
- timestamp
- feature_vector
- data_quality_score

AI Model Inference Layer

The AI model predicts:

Prediction	Example
Triage priority	high risk
Sepsis risk	0.82
Cardiac event probability	0.67

Example output:

Patient ID: 98721

Risk scores:
- Sepsis probability: 0.82
- Cardiac risk: 0.67

Recommended triage level: **High**
Suggested diagnostics: lactate test

Inference pipeline:

feature vector => AI triage model => risk scores

Clinical Decision Support Interface

Emergency Department dashboard

Patient: John Doe
Age: 67

AI alerts

High sepsis risk

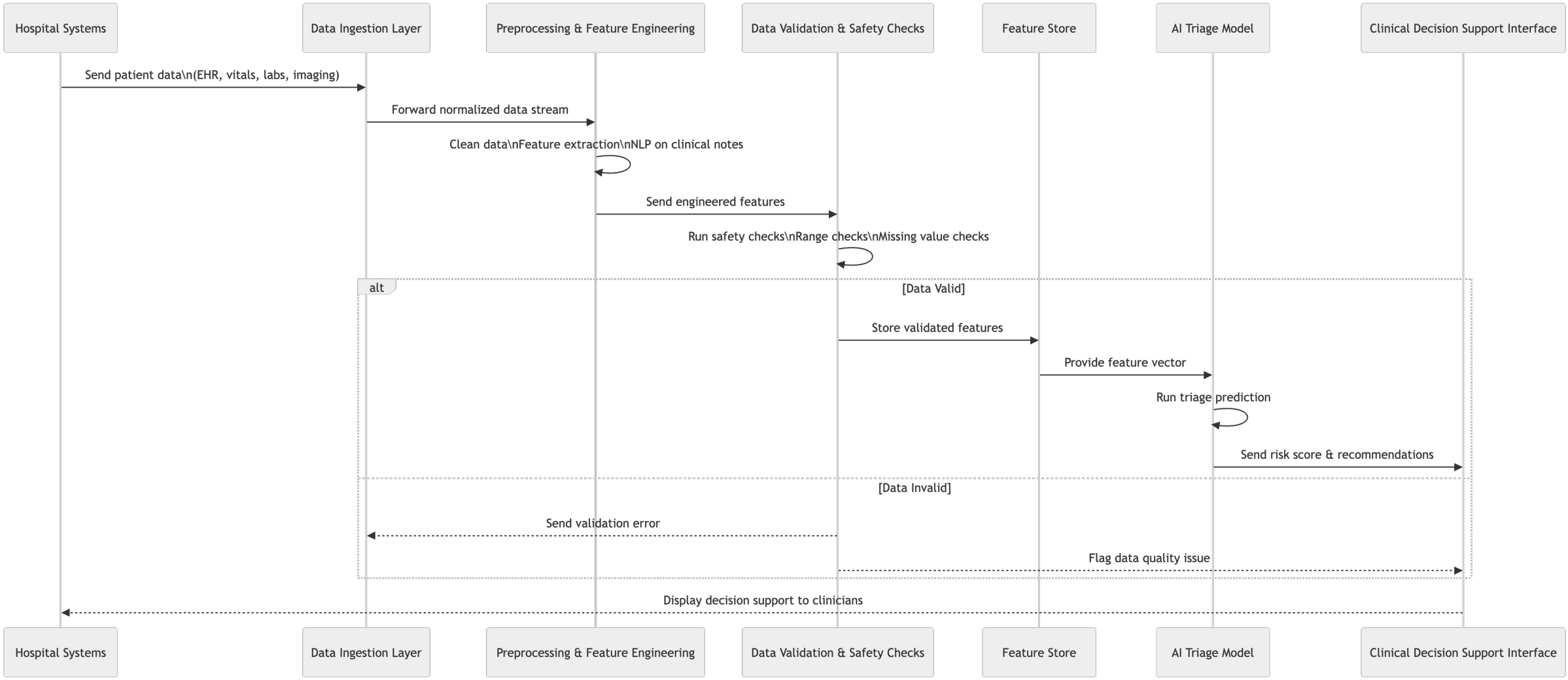
Recommended actions

Order lactate test
Check blood cultures

The clinician **always makes the final decision**.

The clinician always makes the final decision.

AI Data Pipeline for Emergency Department Triage



Failure Scenario 1 — Ingestion Failure

Data lineage & identify

Raw hospital data:

Patient ID: null
Age: 72
Heart Rate: 88
Temperature: 37

Error propagation:

Missing patient ID
↓
Preprocessing
↓
Feature store
↓
AI model prediction
↓
Result cannot be linked to patient

Failure Scenario 2 — Preprocessing Failure

Data lineage & identify

Original data:

Temperature: 39°C

Preprocessing mistake:

Temperature converted to Fahrenheit twice

Result:

Temperature = 182°F

Error propagation:

Incorrect feature

↓

Model interprets as critical fever

↓

False emergency alert

Failure Scenario 3 — Validation Failure

Validation rules are the safety gate of the pipeline

Validation rules are incomplete.

Example corrupted data:

Heart Rate: 500

Error propagation:

Invalid heart rate

↓

Feature store

↓

Model prediction

↓

Extreme risk score

Failure Scenario 1 — Model Failure

distribution shift.

Model trained on **adult data only**.

New patient:

Age: 6
Heart Rate: 120

Error propagation:

Valid data

↓

Model receives unseen population

↓

Incorrect prediction

Closing

This week is about building systems that notice.

Systems that:

- Treat collection as an engineering surface
- Treat preprocessing as a declaration of assumptions
- Treat validation as enforceable contracts
- Treat pipelines as reproducible infrastructure
- Make change visible instead of mysterious

Because clean data isn't enough.
You need pipelines that are:

- **Observable** — so you know when behavior shifts.
- **Reproducible** — so you can explain and debug outcomes.
- **Defensible** — so you can trust decisions built on them.

Thank You